Universidad Modelo

Escuela de Ingeniería

Ing. en Desarrollo de Tec. y Soft.

Asignatura: Estructuras de datos
Grupo: 3ºA Matutino
Entrega: 27/oct/2026

Proyecto #4: SimMarket.$^{2.0}$

Objetivo

Resolver problemas computacionales empleando estructuras de datos lineales.

Modalidad

Equipos.

Instrucciones

Elabora el simulador considerando lo siguiente:

1. La simulación operará bajo el concepto de *iteración*. En cada iteración ocurrirán una serie de eventos por especificar. El programa debe permitir la visualización de cada iteración del algoritmo de simulación, en la cual se deberá mostrar la información completa de los clientes dentro del supermercado, así como el estado actual de las cajas de cobro.

2. En cada iteración, un número aleatorio entre 0 y `NEW_CLIENTS_PER_ITERATION` clientes entrarán al supermercado. Para cada cliente entrante se generarán tres números aleatorios:

   - De 1 a `PRODUCTS_TO_COLLECT`, el número total de artículos a recolectar.
   - De 1 a `ITERATIONS_TO_COLLECT`, el número de iteraciones entre eventos de recolección (disminución del total de artículos a recolectar).
   - De 1 a `ITERATIONS_TO_COLLECT_PRODUCTS`, el número de artículos a disminuir en cada evento de recolección.

   El intervalo entre eventos y la cantidad de artículos a disminuir se generarán una sola vez por cliente y permanecerán fijos durante su recolección. El primer evento de recolección ocurrirá una vez transcurrido el intervalo generado, y los siguientes eventos ocurrirán con la misma periodicidad.

   En cada evento de recolección, el número de artículos pendientes disminuirá según la cantidad generada. Si esta cantidad es mayor que el número de artículos restantes, el número de artículos pendientes se establecerá en cero. El número de artículos pendientes nunca podrá ser negativo.

   Una vez recolectados todos sus artículos, el cliente procederá a hacer fila en las cajas de cobro en la siguiente iteración, no en la misma iteración en la que termine de recolectarlos.

3. Se considerará un total de `TOTAL_CASHIER_LANES` posibles cajas de cobro, de las cuales un número de `EXPRESS_CASHIER_LANES` cajas se denominarán como *cajas rápidas*.

4. Si el número de productos del cliente es menor o igual a `EXPRESS_LANE_MAX_PRODUCTS`, el cliente podrá formarse en una caja rápida. Si el cliente es `Preferente`, también podrá formarse en una caja rápida aunque exceda dicho límite. Todos los clientes podrán formarse en las cajas normales.

   Se deberá establecer la constante `MAX_CUSTOMERS_PER_CASHIER`, que indicará el número máximo de clientes que podrá tener asignada cada caja de cobro, incluyendo al cliente que esté siendo atendido.

   Cuando un cliente termine de recolectar sus productos y corresponda incorporarse a una fila, se evaluarán las cajas elegibles que tengan espacio disponible. El cliente se formará en la caja cuyo tiempo de espera estimado sea menor, considerando tanto los clientes que esperan como el cliente que está siendo atendido.

   Para estimar el tiempo de espera de una caja (j), se sumará el número de iteraciones necesarias para procesar los artículos pendientes de cada cliente que ya se encuentre en su cola:

$$T_j = \sum_{i=1}^{n_j} \left\lceil \frac{P_{ij}}{V_j} \right\rceil$$

   Donde:

   a) (T_j): tiempo de espera estimado para la caja (j), expresado en iteraciones.
   b) (n_j): número de clientes que ya se encuentran en la cola de la caja (j).
   c) (P_{ij}): número de artículos que el cliente (i) tiene pendientes de procesar en la caja.
   d) (V_j): número de artículos que la caja (j) puede procesar por iteración, según la cantidad asignada al inicio de la aplicación.

   Si varias cajas elegibles tienen el mismo tiempo de espera estimado, se priorizará una caja rápida cuando el cliente sea elegible para ella. Si el empate persiste, se elegirá la caja con el menor índice.

   Si una caja está llena, no podrá seleccionarse para recibir nuevos clientes. Si todas las cajas elegibles están llenas, el cliente abandonará el supermercado sin completar su compra. En este caso, deberá eliminarse de la lista de clientes dentro del supermercado y se incrementará un contador de clientes que abandonaron su compra.

5. Al inicio de la aplicación, a cada caja de cobro se le asignará una cantidad de artículos a procesar por iteración entre 1 y `CHECKOUT_PRODUCTS`.

   Sólo se procesarán los artículos del cliente que se encuentre en la primera posición de la cola de cada caja. En cada iteración, una caja procesará artículos únicamente del cliente que se

encuentre al inicio de su cola. Una vez que el cajero procese todos los artículos de ese cliente, este procederá a retirarse de la caja y, por lo tanto, del supermercado. El siguiente cliente de la cola comenzará a ser atendido a partir de la siguiente iteración.

6. Se requiere:
   a) Implementar clases que representen tanto a los clientes como a las cajas de cobro.
   b) Hacer uso de las clases `ArrayQueue` y `ArrayList` desarrolladas en proyectos anteriores.
   c) Implementar una lista de clientes dentro del supermercado.
   d) Implementar una lista de cajas de cobro.
   e) Implementar colas para los clientes formados en las cajas de cobro.
7. Los clientes contarán con una propiedad denominada `Preferente`, bajo la cual, aunque excedan la cantidad `EXPRESS_LANE_MAX_PRODUCTS`, podrán ser atendidos en las cajas rápidas. Esta condición deberá ser aleatoria y poseer una probabilidad de ocurrencia del 10 %.
8. Se deberá establecer la constante `CUSTOMER_ARRIVAL_ITERATIONS`, que indicará el número de iteraciones durante las cuales podrán entrar nuevos clientes al supermercado.

   Una vez transcurrido este período, dejarán de generarse nuevos clientes, pero la simulación continuará ejecutándose hasta que todos los clientes hayan terminado su recorrido o abandonado el supermercado.

   La simulación finalizará únicamente cuando haya terminado el período de llegada de clientes y no queden clientes dentro del supermercado ni clientes en las colas de las cajas de cobro.

Entrega digital

Contenido

Consiste en un solo archivo comprimido ZIP, RAR o 7Z que debe incluir:

1. Directorio completo del proyecto de Visual Studio.
2. Documento (presentación) en formato PDF.

Nomenclatura de archivo

Proyecto_4 - Apellido1_Apellido2_Apellido3_Apellido4.zip (.rar o .7z)

Rúbrica de evaluación

Reporte digital 1 (presentación).

Notas

- ▷ Para iniciar la evaluación todos los integrantes del equipo deben estar presentes.
- ▷ Revisa los requisitos de la entrega digital en la presentación del primer día de clases.